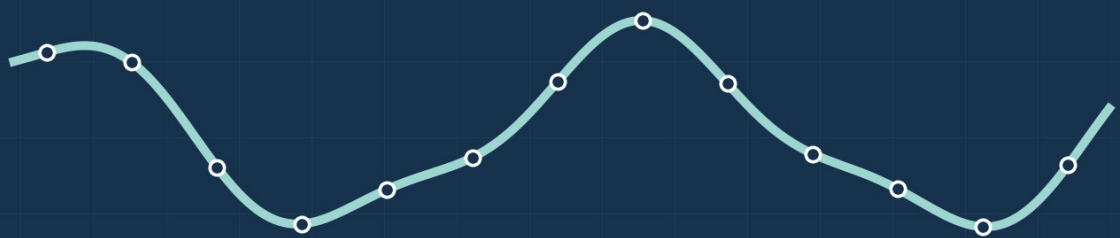
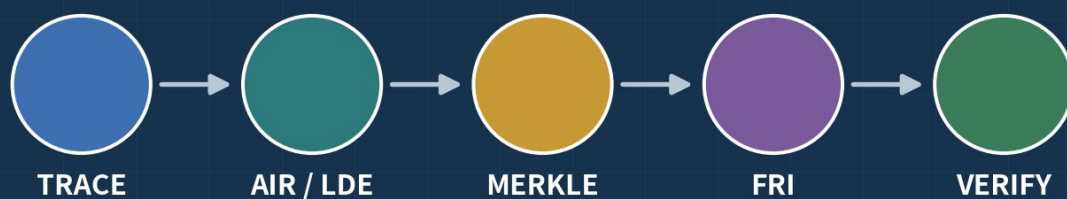


ZK-STARK

図解アトラス

直観から証明・検証・実装まで



Execution Trace / AIR / LDE / Merkle / OODS / DEEP / FRI / Zero-Knowledge

業務説明・研究レビュー向け 2026年8月

はじめに

本資料は、ZK-STARK の構成要素を「処理順の説明」だけでなく、各部品がなぜ必要なのかという因果関係まで視覚化した図解レポートである。ZKP 一般の完全性・健全性・ゼロ知識の基礎説明は最小限とし、実行トレース、AIR、補間、LDE、Merkle コミットメント、制約商、composition polynomial、OODS／DEEP、FRI、ゼロ知識マスキング、実装・デプロイへ焦点を当てる。

図はすべて本資料用に新規作成した概念図である。古典的な乗法部分群上の FRI を直観説明の中心とし、Circle FRI など現代実装との差異は注意欄で明示した。特定ライブラリの証明トランスクリプトを完全に再現するものではなく、方式横断的な理解を目的とする。

15 分で全体像を掴む推奨ルート

図 1 → 図 3 → 図 4 → 図 7 → 図 9 → 図 14 → 図 18 → 図 19 → 図 20 → 図 23

数理構造を追う推奨ルート

図 4～図 19 を順に読み、付録図 A1～A4 で有限体 F17 の数値を確認する。

アプリケーション設計を追う推奨ルート

図 1、図 3、図 20～図 25 を読み、開発者・証明者・検証者・運営者の責任分界を確認する。

図面マップ

セクション	対象図	主題
1. 全体像と直観	図 1～図 3	一枚絵、工場監査の比喻、STARK と ZK-STARK の差
2. 計算から多項式へ	図 4～図 8	実行トレース、AIR、補間、LDE、誤り拡散
3. コミットメントと制約	図 9～図 13	Merkle、Fiat-Shamir、制約商、合成、OODS／DEEP
4. FRI と健全性	図 14～図 18	FRI の責務、folding、query、soundness、四重ロック
5. ゼロ知識とスケーラビリティ	図 19～図 22	マスキング、計算量、proof 構造、パラメータ
6. 実アプリケーション	図 23～図 25	アクター、利用時シーケンス、Groth16 比較
Appendix	付録図 A1～A4	F17 Toy Computation の数値確認

本資料を読む際の前提

- 「低次数」は、対象トレース長とプロトコルが定める次数上限に対して低いという相対的な意味である。
- 「少数照会」は、soundness 目標に応じて複数回行われる。単一の点だけで安全になるという意味ではない。
- ZK-STARK の量子耐性は、ハッシュ関数、パラメータ、Fiat-Shamir 変換を含む具体的な構成に依存する。
- プログラム意味論の正しさは AIR／zkVM プログラムの監査対象であり、証明方式だけでは保証されない。

SECTION 1

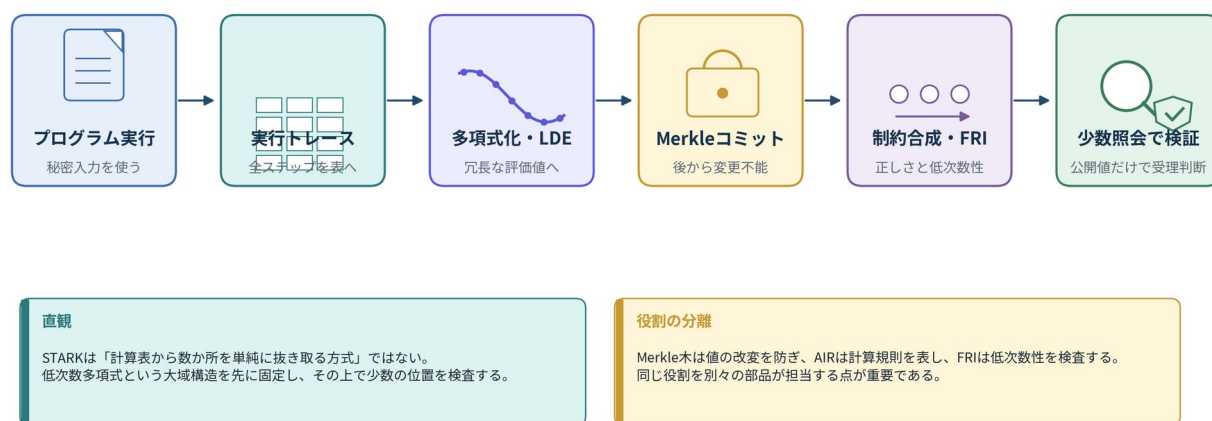
全体像と直観

最初に、ZK-STARK を構成する部品と役割分担を俯瞰する。

図 1 ZK-STARK を一枚で捉える

図1 ZK-STARKを一枚で捉える

計算履歴を「低次数構造を持つ改ざん困難な記録」に変換し、少数照会で全体を検査する



図の読み方 左から右へ、秘密入力を使った計算が実行トレースへ展開され、多項式化・LDE・コミットメント・制約合成・FRIを経て、検証者の少数照会へ到達する。各箱は独立の役割を持ち、前段の出力を後段が検査可能な形へ変換している。

この図で押さえる点

ZK-STARK の核心は、計算全体を低次数多項式という大域構造へ埋め込み、その構造を固定した後でランダムに検査する点にある。

誤解を避けるための注意

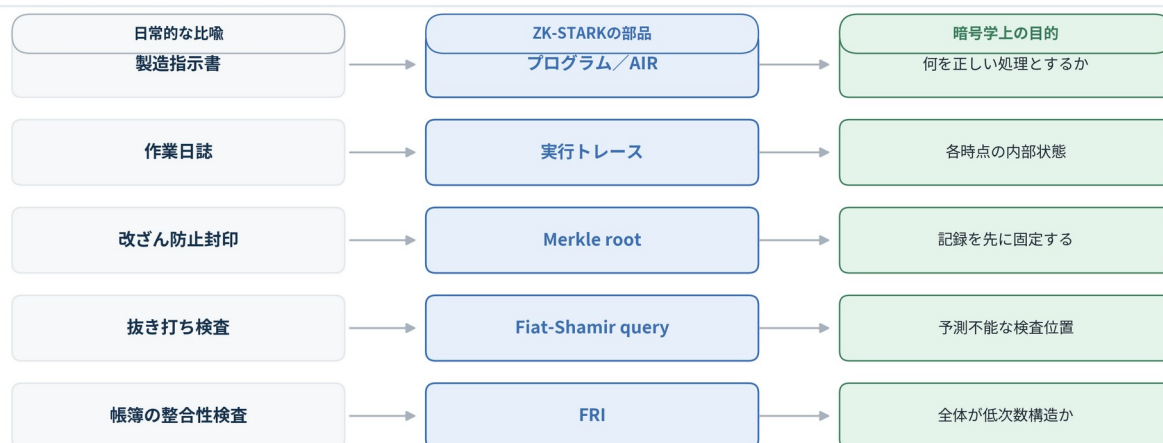
「巨大な表から数行だけを抜き取る」だけではない。低次数性とコミットメントがなければ、少数照会は全体保証にならない。

関連文献：[1][5]

図2 「工場監査」として理解する STARK

図2 「工場監査」として理解するSTARK

比喩は導入用であり、右側が暗号プロトコル上の実体である



比喩の限界

実際には「抜き打ち検査」だけで安全になるのではない。LDEによる冗長性、制約多項式、OODS/DEEP、FRIの低次数検査が組み合わせることで、少数照会が全体保証へ結び付く。

図の読み方 左列の比喩と中央列の暗号部品を対応させて読む。製造指示書がプログラム／AIR、作業日誌がトレース、封印がMerkle root、抜き打ち検査がFiat-Shamir query、帳簿の形の検査がFRIに相当する。

この図で押さえる点

導入段階では「改ざん不能な作業記録を、予告なしに監査する仕組み」と捉えると、登場部品の関係を把握しやすい。

誤解を避けるための注意

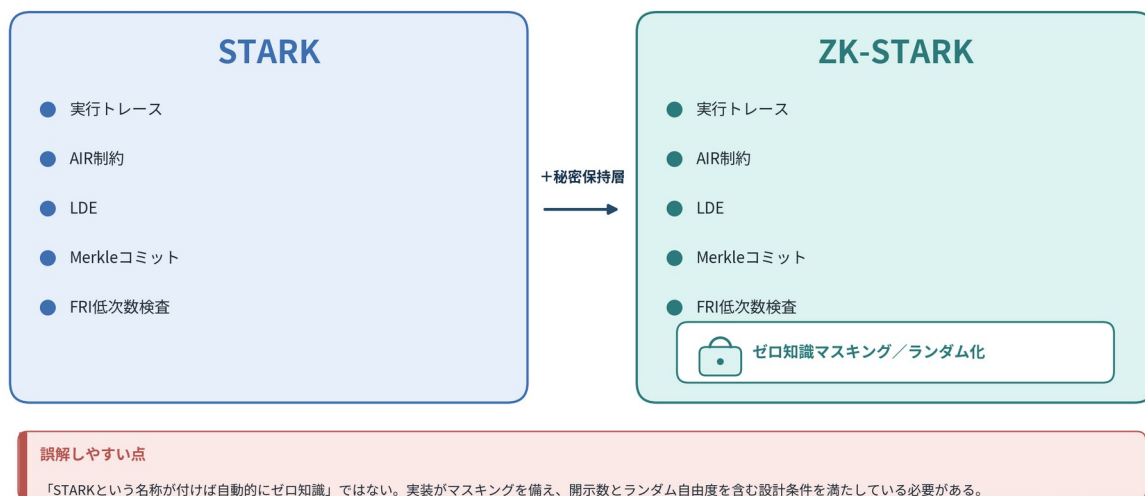
比喩は直観用である。暗号学的健全性は、LDE、制約商、OODS／DEEP、FRIの数理的性質から生じる。

関連文献：[1][5]

図3 STARK と ZK-STARK の違い

図3 STARKとZK-STARKの違い

計算健全性のエンジンに「秘密を隠すランダム化」が加わって初めてゼロ知識になる



図の読み方 左右に共通している部品が計算健全性の基盤である。右側だけに追加されるゼロ知識マスキングが、照会された評価値から秘密トレースが露出することを防ぐ。

この図で押さえる点

STARK は計算の正当性を証明する枠組みであり、ZK-STARK はそれに秘密保持のランダム化を加えた構成である。

誤解を避けるための注意

名称に STARK とあっても、実装が十分なマスキング自由度を備えなければゼロ知識とは限らない。

関連文献：[1][6]

SECTION 2

計算から多項式へ

プログラム実行をトレースへ展開し、低次数多項式と符号的冗長性へ変換する。

図4 プログラム実行を「コマ送りの表」にする

図4 プログラム実行を「コマ送りの表」にする

実行トレースは、各行が時刻、各列がレジスタや中間値に対応する



図の読み方 左のプログラムを一命令ずつ実行し、各時点の x、カウンタ、中間積を中央の表へ書き出す。右側の局所ルールは、現在行と次行の関係だけでプログラムの進行を表している。

この図で押さえる点

実行トレースは、任意の長い計算を同じ種類の局所的な検査へ分解するための共通表現である。

誤解を避けるための注意

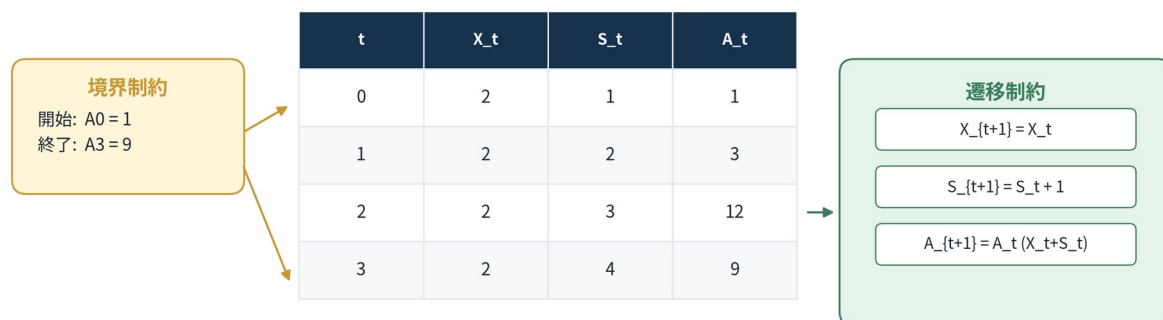
最終値だけが正しくても不十分である。途中の全状態が正しい遷移規則に従う必要がある。

関連文献：[1][4][5]

図5 AIRは「表に対する公開ルールブック」である

図5 AIRは「表に対する公開ルールブック」である

境界制約は開始・終了を固定し、遷移制約は隣り合う行を結ぶ



なぜ局所ルールで十分か

各行が前の行から正しく生成され、最初と最後が固定されていれば、額全体が目的の計算を表す。後段では、この「全行で成立」を多項式の割り切りへ変換する。

図の読み方 中央のトレースに対し、左の境界制約が開始・終了を固定し、右の遷移制約が隣接行の関係を固定する。検証者は同じ AIR を公開ルールとして利用する。

この図で押さえる点

境界制約と遷移制約を組み合わせることで、局所的な確認を目的の計算全体へ結び付けられる。

誤解を避けるための注意

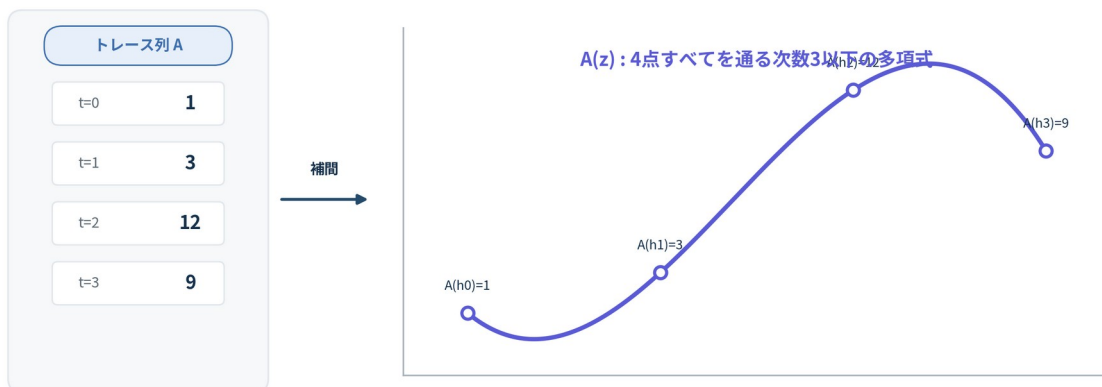
AIR が業務要件を誤って表していれば、ZKP は誤った仕様を正確に証明する。AIR 自体の監査が不可欠である。

関連文献：[1][4]

図6 表の各列を一本の多項式へまとめる

図6 表の各列を一本の多項式へまとめる

補間により、離散的な値が「全体として結び付いた構造」になる



重要

「多項式にした」だけでは正しさは証明されない。後で、AIR制約との整合性と低次数性を別々に検査する必要がある。

図の読み方 左の四つの列値を所定の評価点へ対応させ、それらをすべて通過する低次数多項式 $A(z)$ を作る。点が独立した表要素ではなく、一つの曲線の評価値として結び付く。

この図で押さえる点

補間は、トレース列を代数的検査へ移すための橋である。n 点の列は通常、次数 $n-1$ 以下の多項式で表現できる。

誤解を避けるための注意

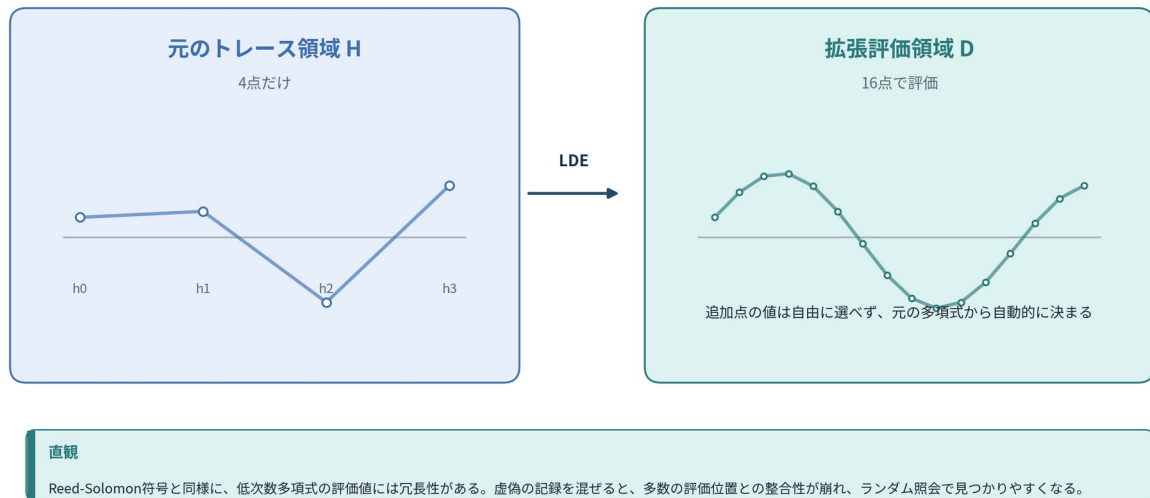
任意の値列にも補間多項式は存在する。低次数であることと AIR を満たすことを、後段で別々に証明しなければならない。

関連文献：[1][5]

図7 LDE は「検査用の冗長性」を加える

図7 LDEは「検査用の冗長性」を加える

同じ低次数多項式を、元のトレース領域より大きな領域で評価する



図の読み方 同じ低次数多項式を、元のトレース点より大きな評価領域 D で計算する。右側の追加点は自由に決める値ではなく、補間多項式が決まれば一意に決まる。

この図で押さえる点

LDE は Reed-Solomon 符号の冗長性を導入し、局所的な虚偽を広い領域の不整合へ変換する。

誤解を避けるための注意

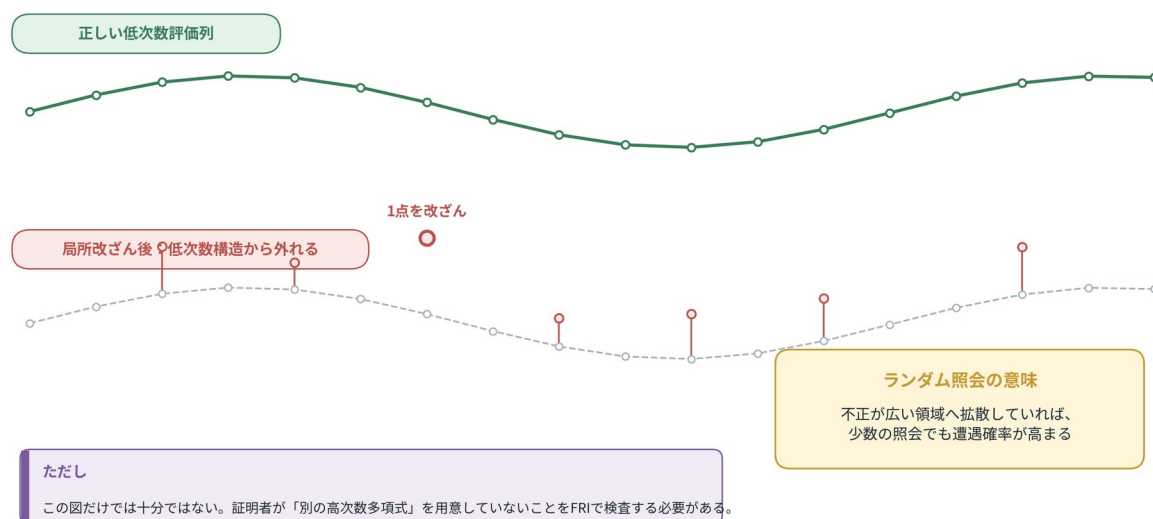
blowup factor を大きくすれば安全性方向に働くが、証明者メモリ、ハッシュ量、証明サイズも増える。

関連文献：[1][2][5]

図8 一か所の嘘が「多数の不整合」へ広がる

図8 一か所の嘘が「多数の不整合」へ広がる

LDEの狙いは、局所的な改ざんを大域的に見つけやすくすることである



図の読み方 上段の正しい評価列に対し、下段では一点の改ざんが低次数曲線との整合性を崩し、他の位置にも不一致が現れる様子を示す。

この図で押さえる点

符号距離の考え方により、不正な評価列を低次数符号語から遠ざけ、ランダム照会で検出しやすくする。

誤解を避けるための注意

高次数多項式なら改ざん点を通せる可能性がある。その可能性を排除する担当がFRIである。

関連文献：[1][2]

SECTION 3

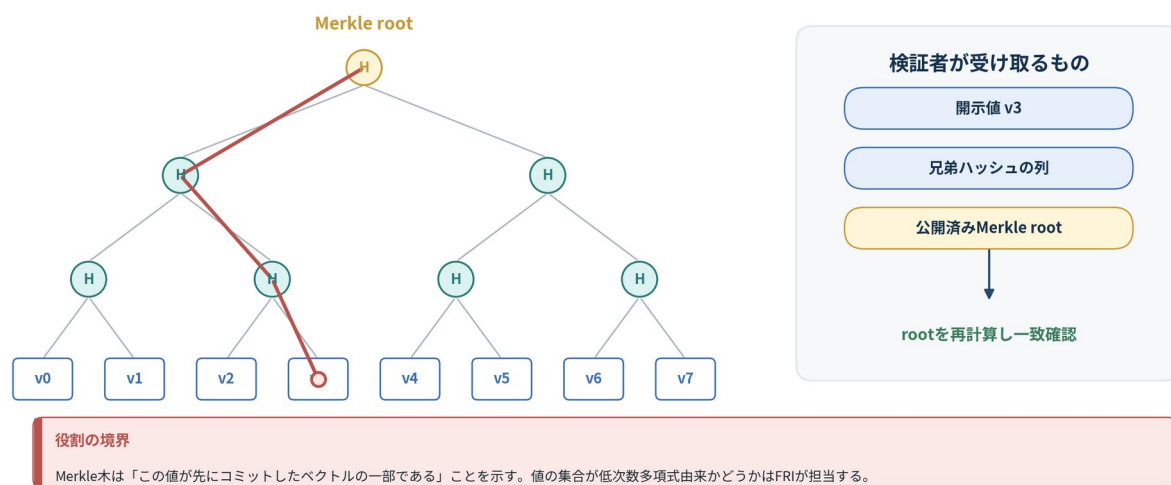
コミットメントと制約

値を事前固定し、計算規則を制約商と composition polynomial へ圧縮する。

図9 Merkle 木は「先に封印し、後で一部だけ開く」

図9 Merkle木は「先に封印し、後で一部だけ開く」

役割はコミットメントであり、低次数性そのものは保証しない



図の読み方 左側で評価値全体から Merkle root を作り、赤線の位置だけを値と authentication path で開示する。検証者は受け取った兄弟ハッシュから root を再計算する。

この図で押さえる点

Merkle コミットメントは、検査位置を知った後の値の差し替えを防ぎ、少数の opening で事前固定を確認できる。

誤解を避けるための注意

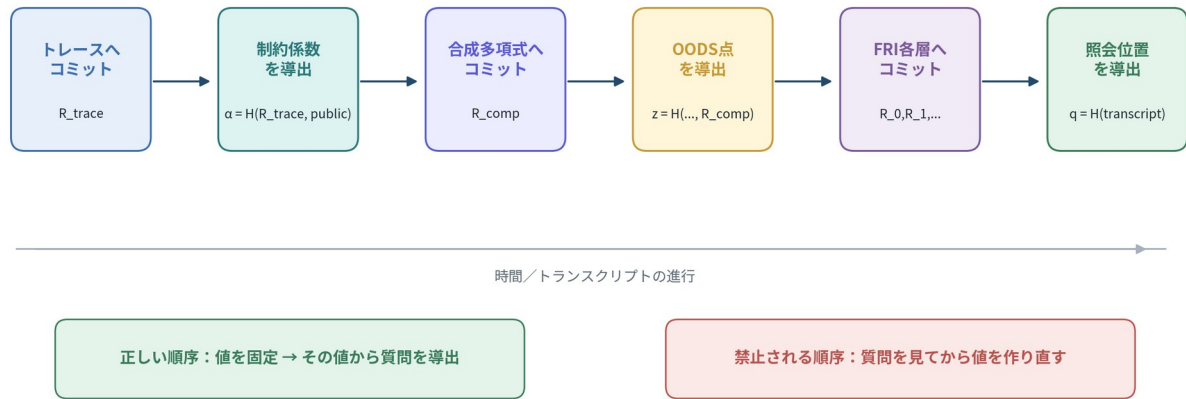
Merkle root が一致しても、その値列が低次数であるとは限らない。低次数性は FRI で確認する。

関連文献：[1][5]

図 10 Fiat-Shamir：コミットしてから質問を決める

図10 Fiat-Shamir：必ず「コミットしてから質問を決める」

非対話型でも、ハッシュにより検証者の予測不能なチャレンジを再現する



図の読み方 各コミットメントをトランスクリプトへ追加した後、そのハッシュから次のランダム係数、OODS点、FRI challenge、query 位置を順番に導出する。

この図で押さえる点

非対話型証明でも、証明者が先に回答を固定し、その後に予測不能な質問が決まる因果関係を再現できる。

誤解を避けるための注意

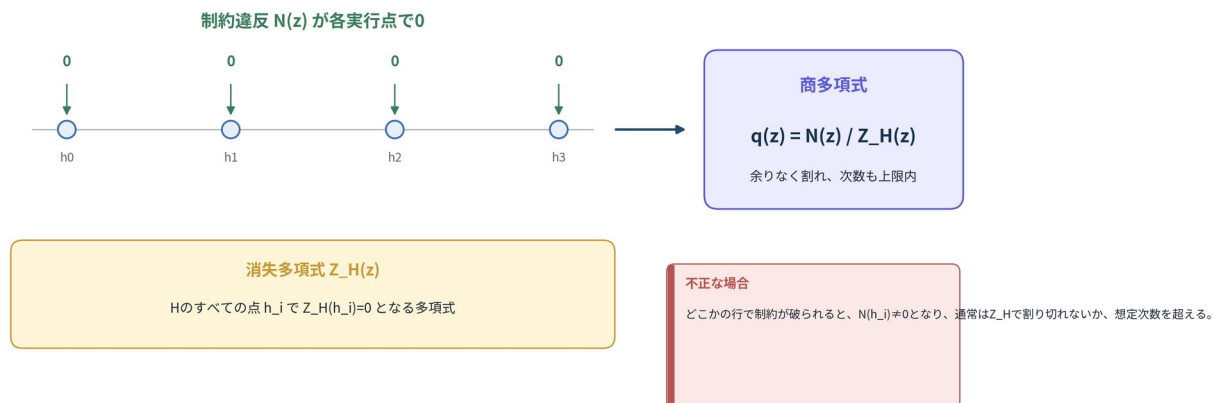
トランスクリプトへ含める順序、ドメイン分離、エンコードが曖昧だとセキュリティ上の欠陥につながる。

関連文献：[1][6]

図 11 AIR 制約を「割り切れるか」という検査へ変換する

図11 AIR制約を「割り切れるか」という検査へ変換する

全実行位置で制約違反が0なら、制約多項式は消失多項式を因数に持つ



図の読み方 制約違反を表す $N(z)$ がすべての実行点 h_i で0なら、同じ点で0となる消失多項式 $Z_H(z)$ を因数に持つ。したがって商 $q(z)=N(z)/Z_H(z)$ が多項式として成立する。

この図で押さえる点

「全行で制約が成立」を、「商多項式が存在し、その次数が上限内」という一つの代数的条件へ変換する。

誤解を避けるための注意

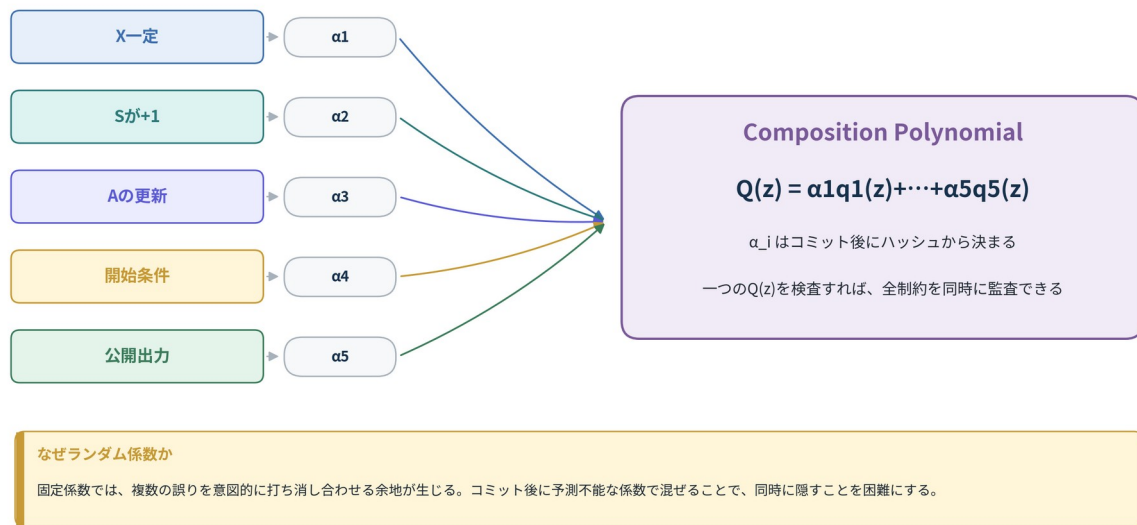
分母が0となるトレース点で直接割るのではなく、恒等式と評価領域外の点を用いて検査する。

関連文献：[1][5]

図 12 多数の制約をランダムに一つへ束ねる

図12 多数の制約をランダムに一つへ束ねる

composition polynomialは、すべての制約を効率よくまとめて検査するための器である



図の読み方 左の複数の制約商 $q_i(z)$ へ、コミット後に得たランダム係数 α_i を掛け、右の composition polynomial $Q(z)$ へ合成する。

この図で押さえる点

一つの $Q(z)$ をコミット・開示・低次数検査することで、多数の AIR 制約を効率的に同時監査できる。

誤解を避けるための注意

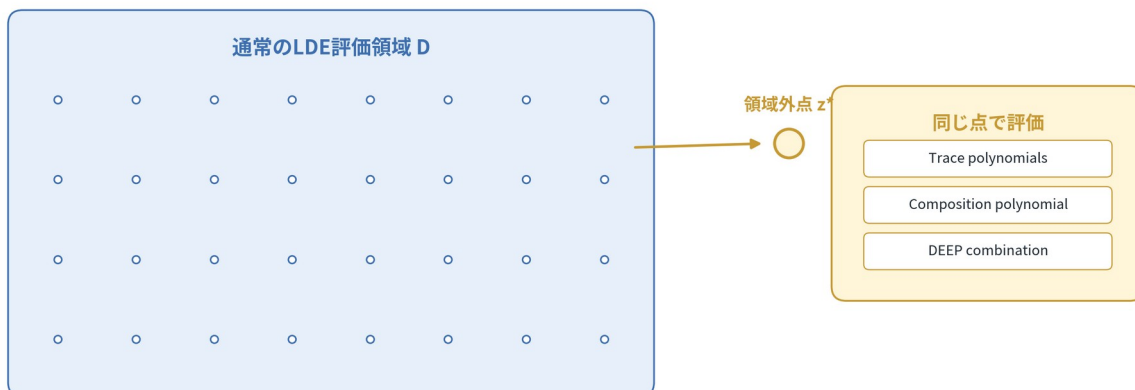
係数が固定または予測可能だと、制約違反同士を相殺する余地が生じる。ランダム性の生成順序が重要である。

関連文献：[1][5][6]

図 13 OODS／DEEP は「用意した格子の外」から照らす

図13 OODS／DEEPは「用意した格子の外」から照らす

評価領域外の予測不能な点で、多項式同士が同じ計算を表すことを強く結び付ける



直観

証明者が「領域Dの中だけで正しく見える複数の候補」を用意していたとしても、コミット後に選ばれる領域外点で同時に整合させることは難しい。

図の読み方 青い LDE 領域 D へコミットした後、領域外のランダム点 z^* を選び、トレース多項式、composition polynomial、DEEP 合成を同じ点で評価する。

この図で押さえる点

領域内だけで整合して見える複数の候補を、予測不能な領域外評価によって一つの多項式関係へ強く結び付ける。

誤解を避けるための注意

OODS は単なる追加サンプリングではなく、リスト復号領域における候補多項式の曖昧さを減らす技法である。

関連文献：[3][6]

SECTION 4

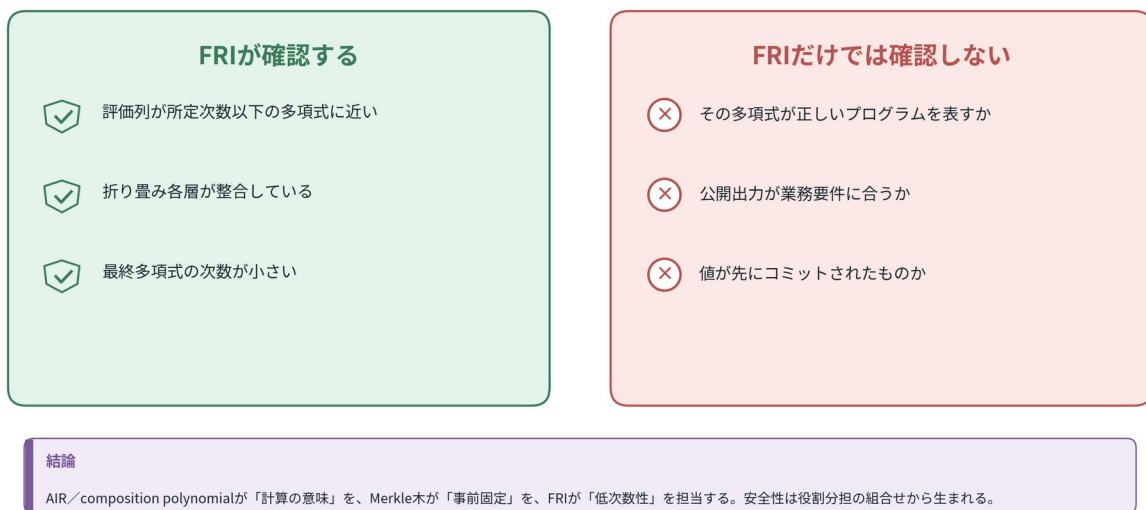
FRI と健全性

低次数性を段階的に検査し、少数照会が全体保証へ結び付く理由を理解する。

図 14 FRI が確認するもの／確認しないもの

図14 FRIが確認するもの／確認しないもの

FRIは計算式の意味を直接理解せず、評価列が低次数多項式に近いかを検査する



図の読み方 左側はFRIの責務、右側はFRI単独では確認しない事項を示す。FRIは評価列の低次数近接性を調べるが、プログラムの意味や事前コミットは別部品が保証する。

この図で押さえる点

FRIを「正しい計算を検証するプロトコル」と一括りにせず、「低次数性の検査器」と理解すると各部品の関係が整理できる。

誤解を避けるための注意

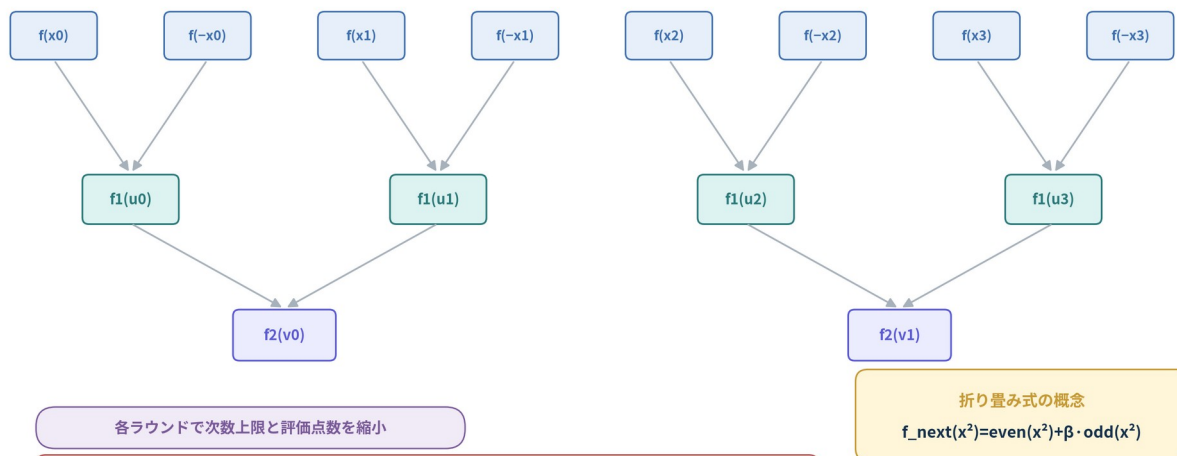
FRIが受理しても、AIRが間違っている、program hashが違う、Merkle openingが不正という問題は別途排除する必要がある。

関連文献：[2][6]

図 15 FRI folding の直観

図15 FRI foldingの直観

古典的な二分foldingでは、 x と $-x$ の評価値をランダム係数で一つへまとめる



注意

現代実装ではfolding arityが2以外の場合や、Circle FRIなど異なる評価領域を用いる場合がある。本図は古典的FRIの直観である。

図の読み方 古典的な二分 folding では、 x と $-x$ の評価値から偶数部・奇数部を取り出し、ランダム係数 β で一つの次層評価値へまとめる。各ラウンドで点数と次数上限が縮小する。

この図で押さえる点

低次数多項式なら folding 後も低次数であり、数ラウンド後には検証者が直接確認できる小さな多項式へ到達する。

誤解を避けるための注意

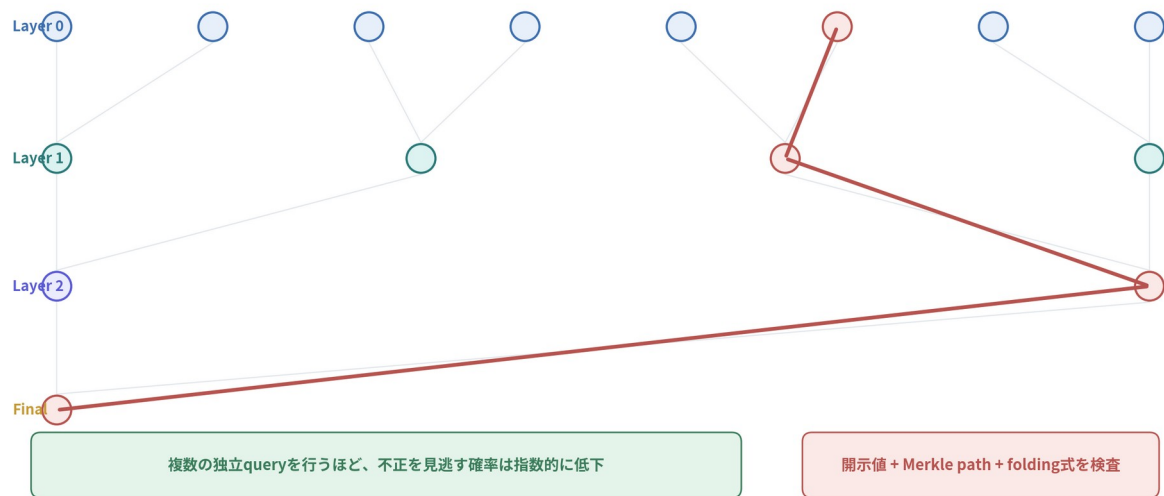
本図は乗法部分群上の古典的 FRI である。Circle FRI や高 arity folding では評価領域と式が異なる。

関連文献：[2][6]

図 16 FRI query は各層を貫く整合経路を調べる

図16 FRI queryは「各層を貫く一本の整合経路」を調べる

検証者は全評価値を読まず、コミット済み各層の対応位置を開示させる



図の読み方 各層の Merkle root が固定された後、赤い query 経路に必要な評価値と Merkle path を開示させる。検証者は層間の folding 式と各 root への包含を同時に確認する。

この図で押さえる点

全層を読み込まず、少数の連鎖した opening だけで、コミットフェーズ全体の整合性を確率的に検査できる。

誤解を避けるための注意

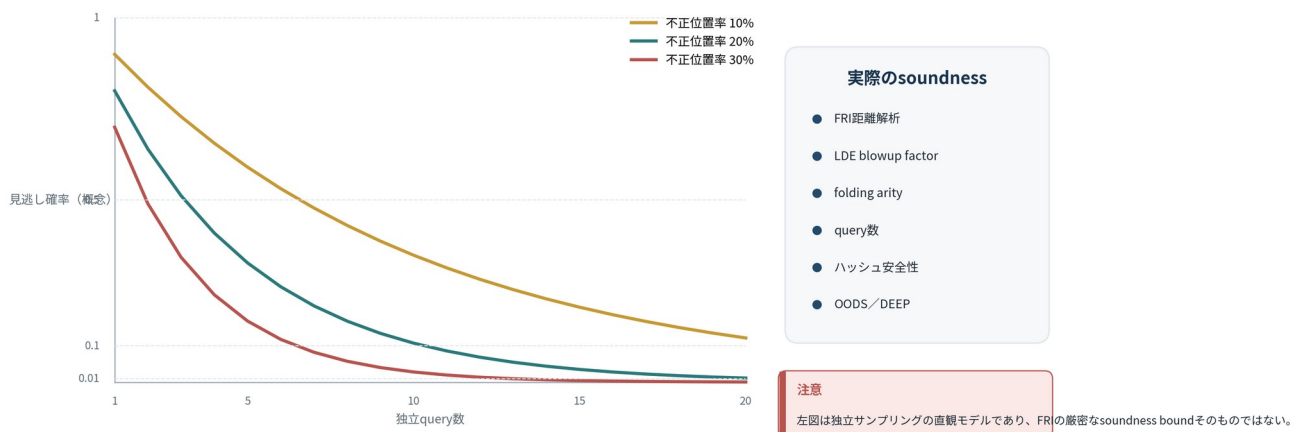
一つの query だけで十分とは限らない。要求 soundness に応じて独立 query 数を設定する。

関連文献：[2][6]

図 17 複数 query による soundness amplification

図17 複数queryによるsoundness amplification

不正な評価列が「悪い位置」を一定割合含むなら、独立照会の反復で見逃し確率が急減する



図の読み方 左の曲線は、不正位置が一定割合あるという単純モデルで、独立 query 数が増えるほど見逃し確率が急減する直観を示す。右側は実際の soundness を左右する要素である。

この図で押さえる点

query 反復は、不正な評価列を偶然見逃す確率を小さくする。セキュリティビット数はパラメータ全体から評価する。

誤解を避けるための注意

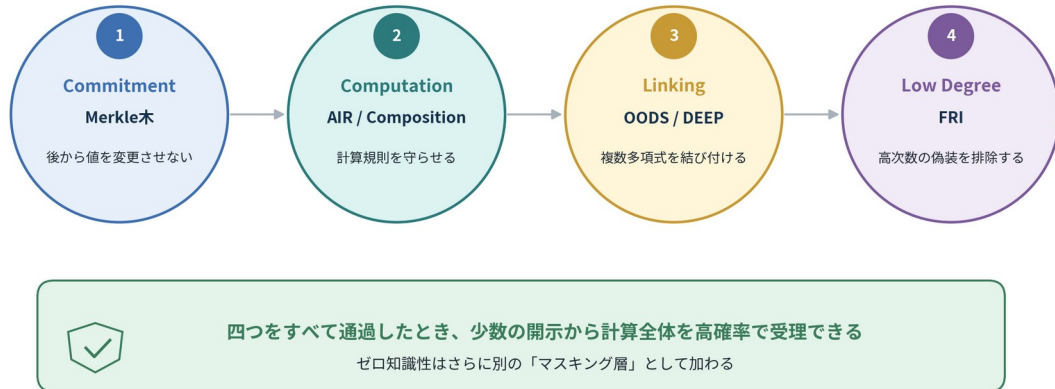
左図は FRI の厳密な soundness bound ではない。実際には符号距離、folding、DEEP、有限体、ハッシュを含む解析が必要である。

関連文献：[2][3]

図 18 STARK の健全性を「四重のロック」で理解する

図18 STARKの健全性は「四重のロック」で理解する

いずれか一つだけでは不十分であり、役割の異なる検査を重ねる



図の読み方 Commitment、Computation、Linking、Low Degree の四段階を左から右へ読む。各ロックは異なる攻撃経路を閉じ、全て通過したときに少数照会が全体保証へ結び付く。

この図で押さえる点

Merkle、AIR/composition、OODS/DEEP、FRI は代替関係ではなく補完関係である。

誤解を避けるための注意

ゼロ知識性はこの四つとは別の性質であり、マスキング層を追加して初めて得られる。

関連文献：[1][2][3]

SECTION 5

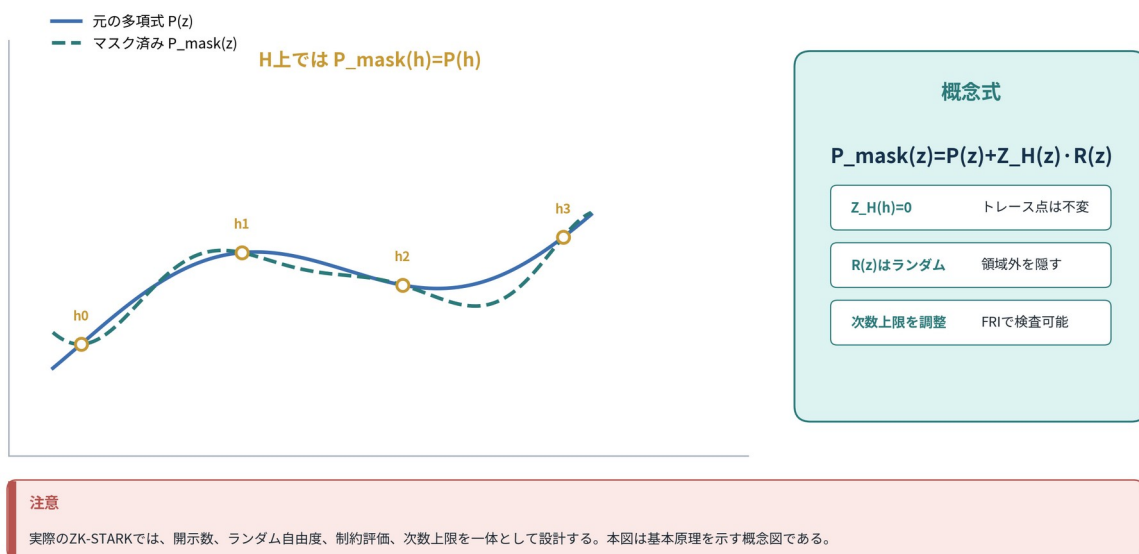
ゼロ知識とスケーラビリティ

秘密保持層、証明者・検証者の計算量、証明データとパラメータ設計を整理する。

図 19 ゼロ知識マスキングの直観

図19 ゼロ知識マスキングの直観

トレース領域上の値を保ったまま、領域外の評価値へランダム性を加える



図の読み方 青の元多項式と緑のマスク済み多項式は、金色のトレース点 h_i では同じ値を取る。一方、領域外ではランダム多項式 $R(z)$ の影響で異なる評価値になる。

この図で押さえる点

消失多項式 $Z_H(z)$ を利用すれば、本来のトレース値を保持しながら、開示される拡張領域の値へランダム性を加えられる。

誤解を避けるための注意

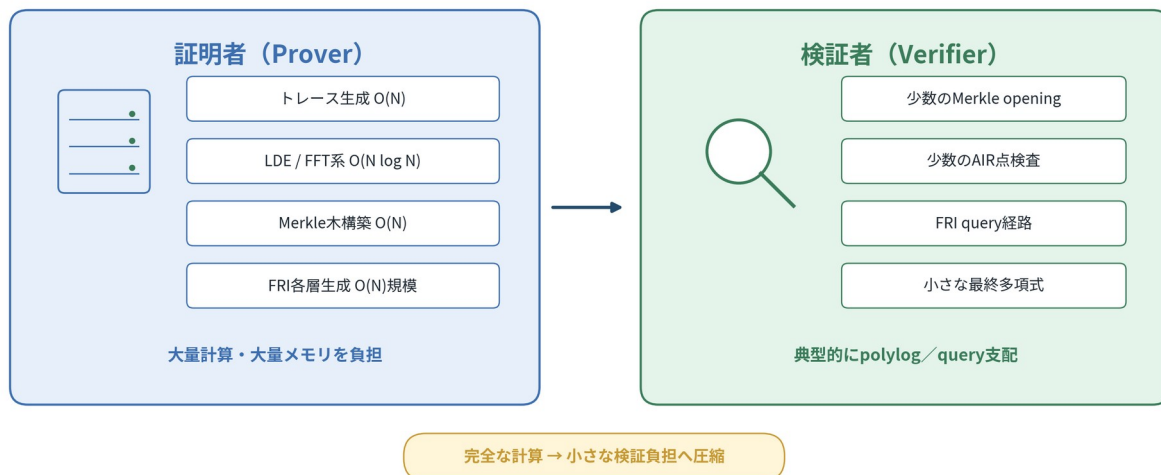
実際の実装では、次数上限、開示数、マスク係数、制約式の整合性を一体として設計する。概念式だけでは安全性を判断できない。

関連文献：[1][6]

図 20 Scalable の意味：重い仕事を証明者へ、検証は少数照会へ

図20 「Scalable」の意味：重い仕事を証明者へ、検証は少数照会へ

計算全体の長さNが増えても、検証者は全トレースを読む必要がない



図の読み方 左は証明者がトレース全体、LDE、Merkle 木、FRI 層を生成する構図であり、右は検証者が少数の opening と局所式だけを確認する構図である。

この図で押さえる点

完全な STARK の証明者処理は一般にトレース長 N に対して準線形規模、検証は polylogarithmic / query 支配となる設計を目指す。

誤解を避けるための注意

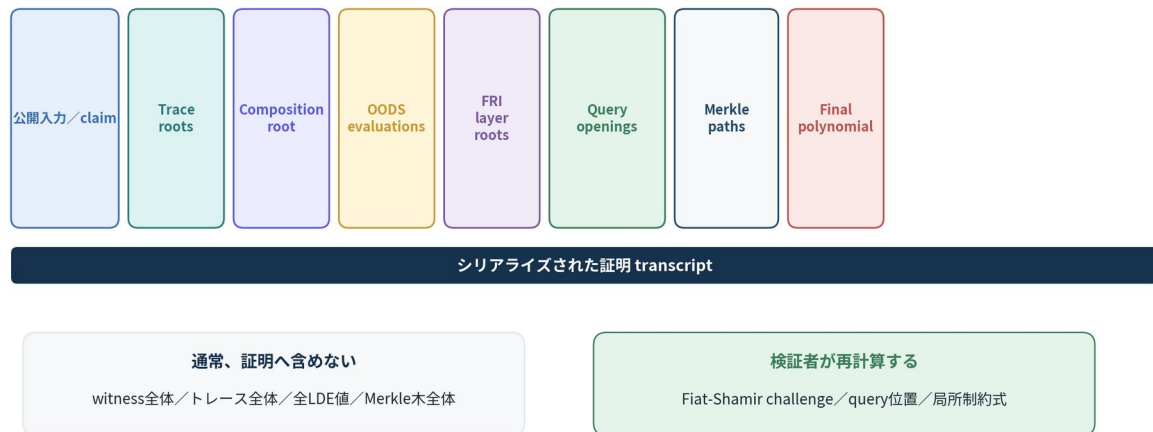
「検証が軽い」代わりに証明生成のメモリとハッシュ処理は重い。小さな計算では固定費が優位性を打ち消す場合がある。

関連文献：[1][2]

図 21 STARK 証明オブジェクトの中身

図21 STARK証明オブジェクトの中身

証明にはトレース全体ではなく、コミットメント・評価値・opening・FRI情報が含まれる



図の読み方 横長の各ブロックが、公開 claim、各種 root、ODS 評価、FRI 情報、query opening、Merkle path、最終多項式を表す。トレース全体は証明へ含まれない。

この図で押さえる点

証明は「全計算記録」ではなく、コミットメントと少数の開示を検証者が再計算可能な順序で束ねたトランスクリプトである。

誤解を避けるための注意

実装ごとにフィールド要素のエンコード、root 数、列のバッチ方法、FRI proof の構造は異なる。

関連文献：[6]

図 22 パラメータは証明サイズ・速度・安全性のつまみである

図22 パラメータは「証明サイズ・速度・安全性」のつまみである

単一の値を最小化するのではなく、システム要件に合わせて全体を設計する

パラメータ	直接効果	主なコスト	主な安全効果
Blowup factor	冗長性 ↑	証明・メモリ ↑	soundness ↑
FRI query数	照会回数 ↑	検証・証明 ↑	見逃し率 ↓
Folding arity	ラウンド数 ↓	1ラウンド開示 ↑	実装依存
Hash function	互換性／速度	回路内コスト	安全性
ZK mask幅	秘密保持 ↑	回数・証明 ↑	開示耐性 ↑

設計原則

「最速のSTARK」ではなく、対象計算、オンチェーン費用、端末性能、証明生成場所、許容soundness error、将来の再帰集約を含めて最適化する。

図の読み方 各行で一つのパラメータが、直接効果、主なコスト、安全効果へどう影響するかを横方向に読む。

この図で押さえる点

blowup、query 数、folding arity、ハッシュ、ZK mask は相互依存するため、システム要件を基に同時最適化する。

誤解を避けるための注意

他方式や他実装のベンチマークを比較する際は、これらのパラメータとセキュリティ水準を揃えなければ公平にならない。

関連文献：[2][3][6]

SECTION 6

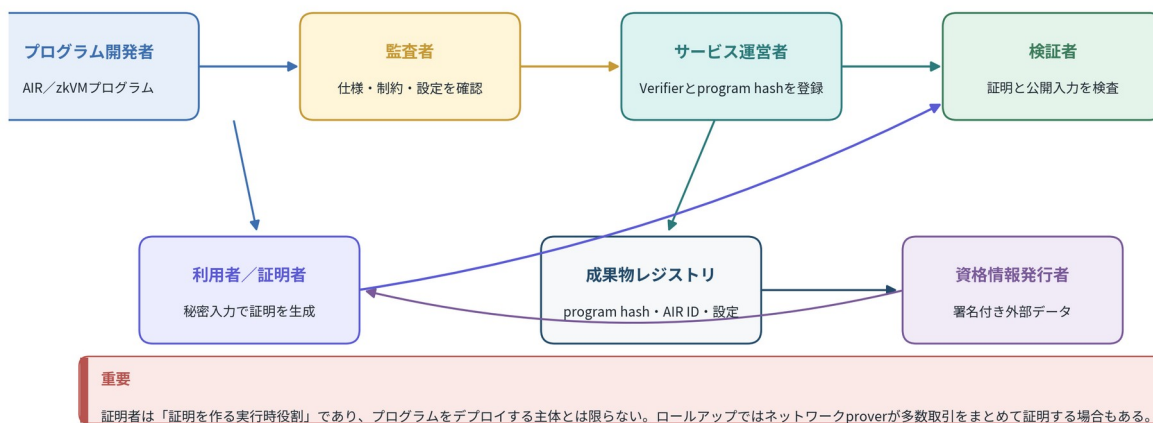
実アプリケーションへの接続

開発・監査・デプロイ・証明・検証の実務的な責任分界を示す。

図 23 ZK-STARK アプリの登場人物と成果物

図23 ZK-STARKアプリの登場人物と成果物

Trusted Setupは不要だが、開発・監査・デプロイ・証明・検証の役割は残る



図の読み方 上段が開発・監査・運用・検証、下段が利用時の証明者、成果物レジストリ、資格情報発行者である。矢印はプログラム、manifest、証明、外部署名データの流れを示す。

この図で押さえる点

Trusted Setup がなくても、プログラム開発、AIR 監査、program hash 管理、Verifier デプロイ、更新管理は必要である。

誤解を避けるための注意

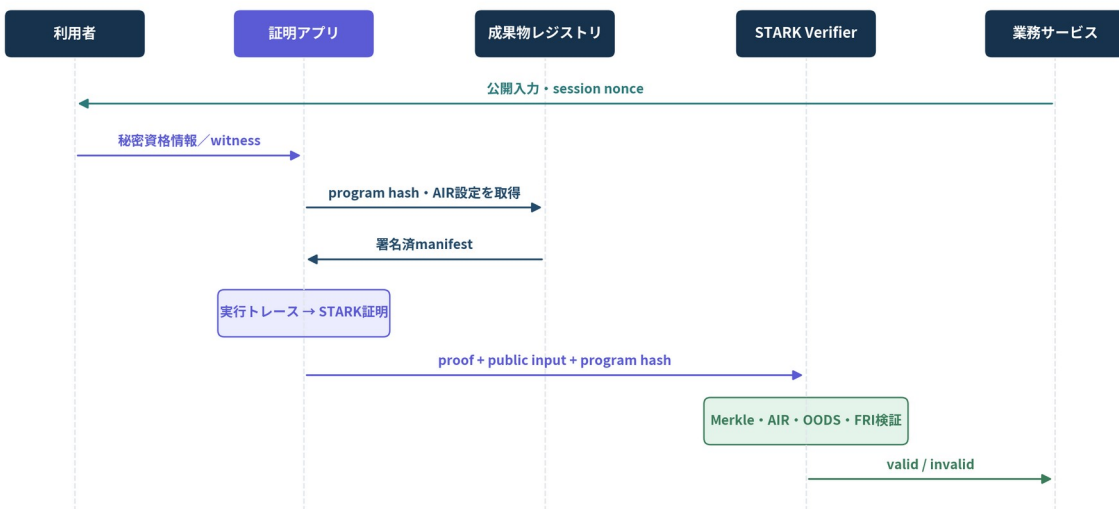
証明者は実行時に proof を作る役割であり、必ずしもプログラム開発者やデプロイヤと同じ主体ではない。

関連文献：[4][6]

図 24 利用時シーケンス：秘密入力からサービス判断まで

図24 利用時シーケンス：秘密入力からサービス判断まで

プログラムハッシュを固定し、公開nonceを証明へ結び付けることで再利用攻撃を防ぐ



プライバシー境界

外部proving serviceへwitnessを送る設計では、そのサービスが新たな秘密情報の受領者となる。可能なら利用者端末内で証明生成する。

図の読み方 業務サービスが nonce を提示し、利用者が秘密情報を証明アプリへ渡す。アプリは承認済み program hash を確認して proof を生成し、Verifier が AIR、Merkle、OODS、FRI を検査する。

この図で押さえる点

program hash で「どのプログラムを証明したか」を固定し、session nonce で証明を現在の要求へ結び付ける。

誤解を避けるための注意

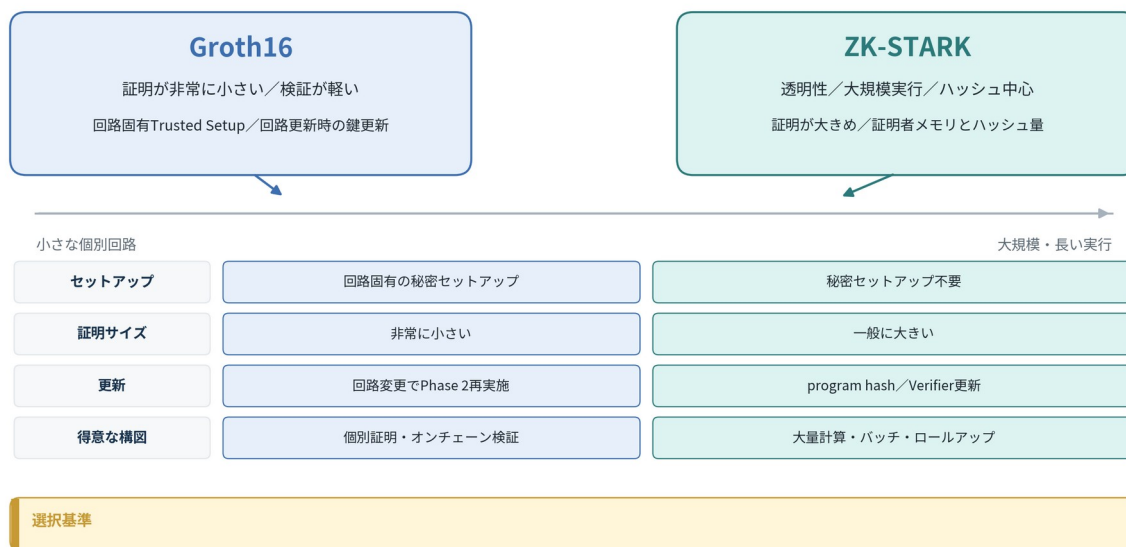
外部 proving service へ witness を送ると、そのサービスが新しい秘密情報の受領者になる。脅威モデル上の境界を明示する必要がある。

関連文献：[4][6]

図 25 Groth16 と ZK-STARK の位置付けを比較する

図25 Groth16とZK-STARKの位置付けを一枚で比較する

優劣ではなく、証明サイズ・セットアップ・計算規模・更新頻度のトレードオフで選択する



同じ「送金」でも、1件ずつの秘匿証明か、数千件の状態遷移集約かで最適方式は変わる。比較では証明関係・ハードウェア・集約件数・セキュリティ水準を揃える。

図の読み方 上段は計算規模の軸、下段はセットアップ、証明サイズ、更新、得意な構図を比較する。方式選定を単純な優劣ではなく、運用条件の違いとして示している。

この図で押さえる点

Groth16 は小さな証明と軽い検証、ZK-STARK は透明性と大規模・反復計算への適合が代表的な強みである。

誤解を避けるための注意

同じ「送金」でも個別取引証明と大量状態遷移集約では比較条件が異なる。証明関係と集約件数を揃える必要がある。

関連文献：[1][7]

SECTION A

Appendix : F17 Toy Computation

小さな有限体上で、トレースから制約商・FRI までの数値を追う。

付録図 A1 Toy Computation : F17 上の実行トレース

付録図A1 Toy Computation : F17上の実行トレース

公開 $y=9$ 、秘密 witness $x=2$ 、関係 $y=(x+1)(x+2)(x+3)$

$$(2+1)(2+2)(2+3)=3 \times 4 \times 5=60 \equiv 9 \pmod{17}$$

t	X_t	S_t	A_t
0	2	1	1
1	2	2	3
2	2	3	12
3	2	4	9

AIR制約

$$X_{t+1}=X_t$$

$$S_{t+1}=S_t+1$$

$$A_{t+1}=A_t(X_t+S_t)$$

$$A_0=1, A_3=9$$

注意

F17は説明用に小さく、公開値 $y=9$ から $x=2$ が特定できるため、実用的な秘密保持例ではない。目的はSTARKの処理構造を手計算で追うことである。

図の読み方 $x=2$ を秘密 witness、 $y=9$ を公開出力として、累積積 A を三回更新する。表と右側の AIR 制約を対応させて読む。

この図で押さえる点

簡単な式も、STARK では各ステップの状態と遷移規則へ展開してから証明対象になる。

誤解を避けるための注意

F17 は小さすぎ、公開値から x が特定できる。暗号的ゼロ知識の安全例ではなく、手計算教材である。

関連文献：[5]および本資料の導出

付録図 A2 Toy Computation：トレース領域と LDE 領域

付録図A2 Toy Computation：トレース領域とLDE領域

$H=\{1,4,16,13\}$ で補間し、 $D=\{3,6,12,7,14,11,5,10\}$ へ拡張評価する



図の読み方 金色のHがトレース4点、緑のDが8点のLDE cosetである。右側は各列の補間多項式と、H上で再現される値を示す。

この図で押さえる点

トレース値を低次数多項式へ補間し、別の大きな領域で評価するという二段階を具体的な有限体で確認できる。

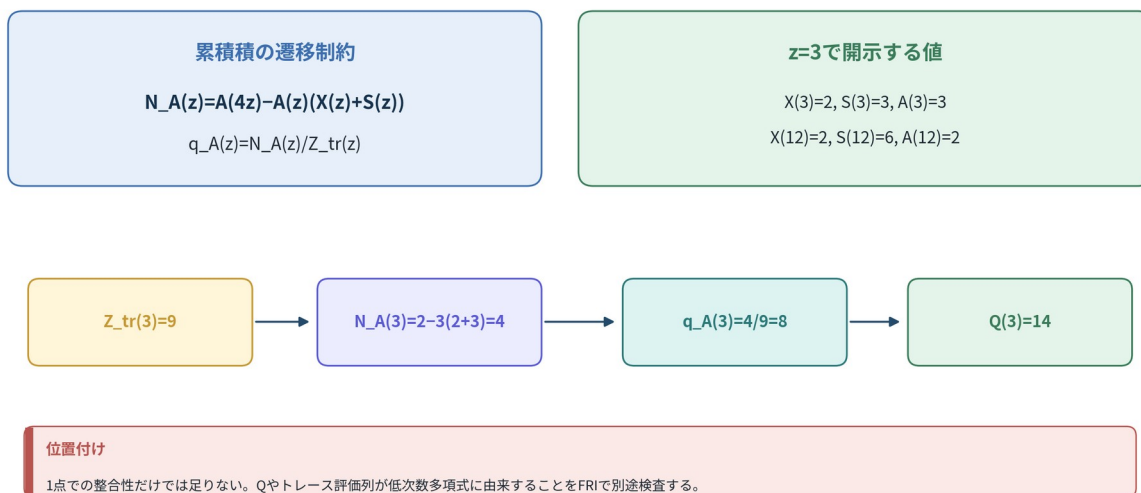
誤解を避けるための注意

円形配置は概念図であり、図上の幾何距離に代数的意味はない。

関連文献：[5]および本資料の導出

付録図 A3 Toy Computation : 制約商と 1 点検査

付録図A3 Toy Computation : 制約商と1点検査

遷移点 $H_{tr}=\{1,4,16\}$ の消失多項式で割り、ランダム点 $z=3$ で整合性を確認する

図の読み方 累積積の遷移制約 $N_A(z)$ を消失多項式 $Z_tr(z)$ で割り、 $z=3$ における開示値から $q_A(3)$ と合成値 $Q(3)$ を再計算する流れである。

この図で押さえる点

検証者は開示された近傍値から局所制約と composition polynomial の一致を計算できる。

誤解を避けるための注意

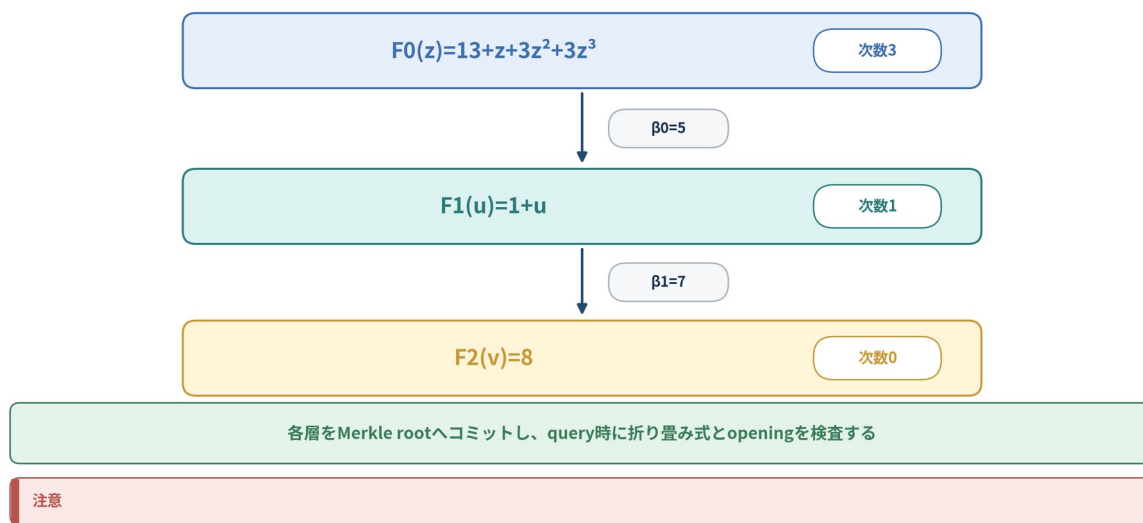
一点の一致だけでは全体は保証できない。多項式の低次数性を FRI で結び付ける必要がある。

関連文献：[5]および本資料の導出

付録図 A4 Toy Computation : FRI folding の数値例

付録図A4 Toy Computation : FRI foldingの数値例

$F_0(z)=13+z+3z^2+3z^3$ を、 $\beta_0=5$ 、 $\beta_1=7$ で折り畳む



図の読み方 次数3の F_0 を $\beta_0=5$ で次数1の F_1 へ、 $\beta_1=7$ で定数 F_2 へ折り畳む。各層の root を先に固定し、後で query に沿って folding 式を検査する。

この図で押さえる点

FRI が大きな低次数性問題を、直接確認できる小さな最終多項式へ段階的に縮小することを数値で追える。

誤解を避けるための注意

この小さな有限体、点数、query 数には暗号的な安全性がない。

関連文献：[2][5]および本資料の導出

用語集

用語	説明
AIR	計算の境界条件と遷移規則を有限体上の多項式制約で表したもの。
Execution Trace	プログラム実行中の状態を時系列で並べた表。
Trace Polynomial	トレース列の値を所定点で再現する補間多項式。
LDE	低次数多項式をより大きな領域で評価して冗長性を加える処理。
Zerofier	所定の点集合ですべて 0 になる消失多項式。
Constraint Quotient	制約違反多項式を消失多項式で割って得る商。
Composition Polynomial	複数の制約商をランダム係数で合成した多項式。
Merkle Commitment	評価値全体を root で固定し、一部を path 付きで開示する仕組み。
Fiat-Shamir	それまでのトランスクリプトのハッシュから非対話的チャレンジを得る方法。
OODS	通常の評価領域外のランダム点で多項式を評価する技法。
DEEP	領域外評価を用いて候補多項式を強く結び付ける技法。
FRI	Reed-Solomon 符号語への近接性、すなわち低次数性を検査する IOPP。
Folding	複数の評価値をランダム係数で次層の評価値へまとめる処理。
Soundness Error	不正な証明を誤って受理する確率の上限。
ZK Masking	検証可能性を保ちながら、照会値から秘密トレースを隠すランダム化。
Program Hash	証明対象となったプログラム／AIR を一意に識別するハッシュ。

参考文献

- [1] E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev, “Scalable, Transparent, and Post-Quantum Secure Computational Integrity,” Cryptology ePrint Archive, Report 2018/046, 2018.
- [2] E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev, “Fast Reed-Solomon Interactive Oracle Proofs of Proximity,” ICALP 2018, LIPIcs 107, Article 14, DOI: 10.4230/LIPIcs.ICALP.2018.14.
- [3] E. Ben-Sasson, L. Goldberg, S. Kopparty, and S. Saraf, “DEEP-FRI: Sampling Outside the Box Improves Soundness,” ITCS 2020, LIPIcs 151, Article 5, DOI: 10.4230/LIPIcs.ITCS.2020.5.
- [4] L. Goldberg, S. Papini, and M. Riabzev, “Cairo - a Turing-complete STARK-friendly CPU Architecture,” Cryptology ePrint Archive, Report 2021/1063, revised 2025.
- [5] StarkWare, “STARK 101: Write a STARK Prover from Scratch,” tutorial materials, accessed August 2026.
- [6] Starknet Documentation, “S-two Book: STARK Proof Generation and Verification; AIR to Composition Polynomial; Circle FRI,” accessed August 2026.
- [7] J. Groth, “On the Size of Pairing-based Non-interactive Arguments,” EUROCRYPT 2016, LNCS 9666, pp. 305-326.

図面の利用について

本資料の図面は説明用に新規作成したオリジナル図である。SVG および PNG 版を付属パッケージに収録している。特定実装の仕様書へ転用する際は、当該実装の有限体、評価領域、FRI 方式、トランスクリプト順序を一次資料で再確認すること。